

Inteligência Artificial 2018/2019

Ficha de Exercícios 9: Processamento de Linguagem Natural

9. Processamento de Linguagem Natural (DCGs)

9.1 Interpretação de frases declarativas (DCG)

Escreva um programa para interpretar frases declarativas. Se a frase estiver correta, o programa deve indicar a concordância ou não com a afirmação efetuada.

Se a frase estiver incorreta, o programa deve indicar se se trata de um erro sintático ou semântico. As frases a analisar são do tipo:

os rapazes jogam futebol	Resposta: concordo
o rui gosta de morangos	Resposta: discordo
o elefante joga futebol	Resposta: erro semântico
os rapazes joga futebol	Resposta: erro de sintaxe

Especifique a base de conhecimento.

9.2 - Interpretação de frases interrogativas (DCG)

Escreva um programa para interpretar frases interrogativas. O programa deve detetar erros sintáticos e semânticos.

Se a frase estiver correta, deve ser apresentada a resposta à questão efetuada. As frases a analisar são do tipo:

quem gosta de morangos	Resposta: [luis, rita, ana]
quantas pessoas jogam futebol	Resposta: 3
quais sao os humanos que gostam de morangos	Resposta: [luis,rita,ana]
quem jogam futebol	Resposta: erro de sintaxe

Especifique a base de conhecimento.

Solução 9.1 e 9.2

```
inicio :-
    write('Escreva uma frase: '), nl,
    read(Frase), % ler frase entre plicas '...'
    transf_lista(Frase, Lista_Pal), % converter em lista de palavras
    verifica(Lista_Pal).

% conversão de frase em lista de palavras
transf_lista(Frase, Lista_Pal) :-
    string_to_list(Frase, Lista_char),
    faz_palavras(Lista_char, [], Lista_Pal).

faz_palavras([], L_char1, [Pal1]) :-
    atom_chars(Pal1, L_char1).
faz_palavras([32|T], L_char1, [Pal1|T1]) :-
    atom_chars(Pal1, L_char1),
    faz_palavras(T, [], T1).
faz_palavras([X|T], L_char, L_Pal) :-
    append(L_char, [X], L_char1),
    faz_palavras(T, L_char1, L_Pal).
```

%%%%%%%%%

```

% Versão simplificada, sem obter a árvore de derivação
%%%

%%%
%%% só verificar sintaxe
%%%

% verifica a frase passada como uma lista de palavras
verifica_0(ListaPal) :-
    frase_0(ListaPal, []) ; write('erro de sintaxe').

% frase afirmativa
frase_0 --> sintagma_nominal_0(N), sintagma_verbal_0(N).
                                     % concordância em número (singular/plural)

sintagma_nominal_0(N) --> determinante(N-G, _), nome(N-G, _, _).
                                     % concordância em número e género (masc./fem.)
sintagma_nominal_0(N) --> nome(N-, _, _).

sintagma_verbal_0(N) -->
    verbo(N, _, _, gostar), preposicao(N1-G1, _), nome(N1-G1, _, _).
sintagma_verbal_0(N) -->
    verbo(N, _, _, _), sintagma_nominal_0(_).

%%%
%%% verificar sintaxe e confirmar pela base de dados
%%%

% verifica a frase passada como uma lista de palavras
verifica_1(ListaPal) :-
    frase_1(ListaPal, []) ; write('erro de sintaxe').

% frase afirmativa
frase_1 -->
    sintagma_nominal_1(N, Sujeito),
    sintagma_verbal_1(N, Accao, Objecto),
    % frase sintaticamente bem construída: verifica se
    % a afirmação condiz com a base de dados
    { concorda_frase(Accao, Sujeito, Objecto), nl }.

sintagma_nominal_1(N, Sujeito) --> determinante(N-G, _), nome(N-G, _, Sujeito).
sintagma_nominal_1(N, Sujeito) --> nome(N-, _, Sujeito).

sintagma_verbal_1(N, gostar, Obj) -->
    verbo(N, _, _, gostar), preposicao(N1-G1, _), nome(N1-G1, _, Obj).
sintagma_verbal_1(N, Accao, Obj) -->
    verbo(N, _, _, Accao), sintagma_nominal_1(_, Obj).

%%%
%%% verificar sintaxe e semântica e confirmar pela base de dados
%%%

% verifica a frase passada como uma lista de palavras
verifica_(ListaPal) :-
    ( retract(erro_semantico) ; true ),
    ( frase_(ListaPal, [])
    ; ( erro_semantico, write('erro semantico') )
    ; write('erro de sintaxe') ).

% frase afirmativa
frase_ -->
    sintagma_nominal_(N, Sujeito),
    % Sujeito serve para verificação semantica: concordância actor/acção (verbo)
    sintagma_verbal_(N, Sujeito, Accao, Objecto),
    % frase bem construída: verifica se a afirmação condiz com a base de dados
    { concorda_frase(Accao, Sujeito, Objecto), nl }.

sintagma_nominal_(N, Sujeito) --> determinante(N-G, _), nome(N-G, _, Sujeito).
sintagma_nominal_(N, Sujeito) --> nome(N-, _, Sujeito).

```

```

sintagma_verbal_(N, Nome, gostar, Obj) --> verbo(N, _, Nome, gostar),
preposicao(N1-G1, _), nome(N1-G1, _, Obj).
sintagma_verbal_(N, Nome, Accao, Obj) --> verbo(N, _, Nome, Accao),
sintagma_nominal_(_, Obj).

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

% Versão obtendo a árvore de derivação (e mais alguns melhoramentos)
%%%

```

```

% verifica a frase passada como uma lista de palavras
verifica(ListaPal) :-

```

```

    ( retract(erro_semantico) ; true ),
    ( ( frase(Estrutura, ListaPal, []),
        write(Estrutura) )
      ; ( erro_semantico, write('erro semantico') )
      ; write('erro de sintaxe') ).

```

```

% frase afirmativa

```

```

frase(f(SN, SV)) -->
    sintagma_nominal(SN, N, Sujeito),
    sintagma_verbal(SV, N, Sujeito, Accao, Objecto),
    % frase bem construída: verifica se a afirmação condiz com a base de dados
    { concorda_frase(Accao, Sujeito, Objecto), nl }.

```

```

sintagma_nominal(SN, p, [Sujeito1,Sujeito2]) -->

```

```

    sintagma_nom1(SN1, N1, Sujeito1), [e],
    sintagma_nom1(SN2, N2, Sujeito2),
    { SN1 =.. [_|L1], SN2 =.. [_|L2],
      append(L1, [e|L2], L),
      SN =.. [sn|L] }.

```

```

sintagma_nominal(SN, N, Sujeito) -->    % não havendo "e" no SN
    sintagma_nom1(SN, N, Sujeito).

```

```

sintagma_nom1(sn(det(D),nome(Nome)), N, Sujeito) -->

```

```

    determinante(N-G, D),
    nome(N-G, Nome, Sujeito).

```

```

sintagma_nom1(sn(nome(Nome)), N, Sujeito) -->

```

```

    nome(N-, Nome, Sujeito).

```

```

sintagma_verbal(sv(verbo(V),SN), N, Nome, gostar, Obj) -->

```

```

    % "gostar" é especial (preposição a seguir, e.g. "de")

```

```

    verbo(N, V, Nome, gostar),
    preposicao(N1-G1, P),
    nome(N1-G1, Nome1, Obj),
    { SN=sn(preop(P), nome(Nome1)) }.

```

```

sintagma_verbal(sv(verbo(V),SN), N, Nome, Accao, Obj) -->

```

```

    verbo(N, V, Nome, Accao),
    sintagma_nominal(SN, _, Obj).

```

```

% verifica se a afirmação condiz com a base de dados

```

```

concorda_frase(_, [], _) :- write(concordo).

```

```

concorda_frase(Accao, [Sujeito1|OSuj], Objecto) :-

```

```

    ( Predicado =.. [Accao, Sujeito1, Objecto], Predicado,
      concorda_frase(Accao, OSuj, Objecto) )
    ; write(discordo).

```

```

concorda_frase(Accao, Sujeito, Objecto) :-    % não havendo "e" no SN

```

```

    ( ( Predicado =.. [Accao, Sujeito, Objecto], Predicado, write(concordo) )
      ; write(discordo) ).

```

```

%%%

```

```

% gramática

```

```

%

```

```

% determinante: - 2º argumento é para a construção da árvore de derivação

```

```

determinante(s-m, o) --> [o].

```

```

determinante(p-m, os) --> [os].

```

```

determinante(s-f, a) --> [a].

```

```

% preposicao: - 2º argumento é para a construção da árvore de derivação
preposicao(_, de) --> [de].
preposicao(s-f, da) --> [da].

% nome:
% - 2º argumento é para a construção da árvore de derivação
% - 3º argumento é para a consulta à BD (identifica o sujeito/objecto)
nome(p-m, rapazes, rapaz) --> [rapazes].
nome(s-m, rapaz, rapaz) --> [rapaz].
nome(s-m, rui, rui) --> [rui].
nome(s-m, luis, luis) --> [luis].
nome(s-f, rita, rita) --> [rita].
nome(s-f, ana, ana) --> [ana].
nome(s-f, maria, maria) --> [maria].
nome(s-m, elefante, elefante) --> [elefante].
nome(p-m, caes, cao) --> [caes].
nome(p-m, gatos, gato) --> [gatos].
nome(s-m, cao, cao) --> [cao].
nome(s-m, gato, gato) --> [gato].
nome(s-m, futebol, futebol) --> [futebol].
nome(p-m, morangos, morango) --> [morangos].
nome(p-m, amendoins, amendoim) --> [amendoins].
nome(p-m, bolachas, bolacha) --> [bolachas].
nome(p-m, humanos, humano) --> [humanos].
nome(p-f, pessoas, humano) --> [pessoas].

% verbo:
% - 2º argumento é para a construção da árvore de derivação
% - 3º argumento é para validação semântica
% - 4º argumento é para a consulta à BD (identifica a acção, que será um
predicado na BD)
verbo(s, joga, Sujeito, jogar) --> [joga], { humano(Sujeito) ;
(assert(erro_semantico), fail) }.
verbo(p, jogam, Sujeito, jogar) --> [jogam], { humano(Sujeito) ;
(assert(erro_semantico), fail) }.
verbo(s, gosta, _, gostar) --> [gosta].
verbo(p, gostam, _, gostar) --> [gostam].
verbo(s, come, _, comer) --> [come].
verbo(p, comem, _, comer) --> [comem].
verbo(p, sao, _, ser) --> [sao].

%%%
% base de dados
%

humano(rapaz).
humano(rui).
humano(maria).
humano(rita).
humano(ana).
humano(luis).
humano(humano).
humano([]).
humano([H|T]) :- humano(H), humano(T).

jogar(rapaz, futebol).
jogar(rui, futebol).
jogar(pokemon, futebol).

gostar(luis, morango).
gostar(rita, morango).
gostar(ana, morango).
gostar(rui, maria).
gostar(caos, bolacha).
gostar(gato, bolacha).

comer(elefante, amendoim).

ser(rui, rapaz).
ser(X, humano) :- humano(X).

```

% 9.2

%%

% Versão simplificada, sem obter a árvore de derivação

%%%

%%%

%% só verificar sintaxe

%%%

% frase interrogativa

frase_0 --> sintagma_interrogativo_0(N), sintagma_verbal_0(N). % concordância em número (singular/plural)

sintagma_interrogativo_0(N) --> pron_inter(N-G, _, _), sintagma_nom_inter_0(N, G).

% concordância em número e género (masc./fem.)

sintagma_interrogativo_0(N) --> pron_inter(N-, _, _).

sintagma_nom_inter_0(N, G) --> % 'quais os humanos que...' em vez de 'quais são os humanos que...'

determinante(N-G, _), nome(N-G, _, _), pronome(_, que). % concordância em número e género (masc./fem.)

sintagma_nom_inter_0(N, G) --> nome(N-G, _, _).

%%%

%% verificar sintaxe e confirmar pela base de dados

%%%

% frase interrogativa

frase_1 -->

sintagma_interrogativo_1(Q, N, Atributo),

sintagma_verbal_1(N, Accao, Objecto),

% frase sintacticamente bem construída: procura resposta na base de dados

{ responde(Q, Atributo, Accao, Objecto) }.

sintagma_interrogativo_1(Q, N, Atributo) --> pron_inter(N-G, _, Q),

sintagma_nom_inter_1(N, G, Atributo).

sintagma_interrogativo_1(Q, N, _) --> pron_inter(N-, _, Q).

sintagma_nom_inter_1(N, G, Atributo) -->

% 'quais os humanos que...' em vez de 'quais são os humanos que...'

determinante(N-G, _), nome(N-G, _, Atributo), pronome(_, que).

sintagma_nom_inter_1(N, G, Atributo) -->

nome(N-G, _, Atributo).

%%

% Versão obtendo a árvore de derivação (e mais alguns melhoramentos)

%%%

% frase interrogativa

frase(f(SI, SV)) -->

sintagma_interrogativo(SI, Q, N, Atributo),

sintagma_verbal(SV, N, _, Accao, Objecto),

% frase bem construída: procura resposta na base de dados

{ responde(Q, Atributo, Accao, Objecto) }.

sintagma_interrogativo(pi(I), Q, N, Atributo) --> pron_inter(N-G, I, Q),

sintagma_nom_inter(SNI, N, G, Atributo).

sintagma_interrogativo(pi(I), Q, N, _) --> pron_inter(N-, I, Q).

sintagma_nom_inter(sn(verbo(V),det(D),nome(Nome),p(que)), N, G, Atributo) -->

(verbo(N, V, _, ser) ; { true }),

determinante(N-G, D),

nome(N-G, Nome, Atributo),

pronome(_, que).

sintagma_nom_inter(sn(nome(Nome)), N, G, Atributo) -->

```

nome(N-G, Nome, Atributo).

responde(Q, Atributo, Accao, Objecto) :-
    var(Atributo), !,
    Predicado =.. [Accao, Sujeito, Objecto],    % para obter sujeitos desta
acção/objecto
    findall(Sujeito, Predicado, Lista),
    ( ( Q = qual, write(Lista) )
    ; ( length(Lista, Nlista), write(Nlista) )
    ), nl.
responde(Q, Atributo, Accao, Objecto) :-
    Predicado =.. [Accao, Sujeito, Objecto],    % para obter sujeitos desta
acção/objecto ...
    Predicado1 =.. [ser, Sujeito, Atributo],    % ... que têm este atributo
    findall(Sujeito, (Predicado, Predicado1), Lista),
    ( ( Q = qual, write(Lista) )
    ; ( length(Lista, Nlista), write(Nlista) )
    ), nl.

%%%
% gramática
%

% pronome_inter:    - 2º argumento é para a construção da árvore de derivação
%                  - 3º argumento é para a consulta à BD (identifica se a resposta
deve ser qualitativa ou quantitativa)
pron_inter(_-, quem, qual) --> [quem].
pron_inter(p-, quais, qual) --> [quais].
pron_inter(p-m, quantos, quant) --> [quantos].
pron_inter(p-f, quantas, quant) --> [quantas].

% pronome:          - 2º argumento é para a construção da árvore de derivação
pronome(_, que) --> [que].

```